ZCode 自定义工作体系

ZCode 自定义 AI 工作体系说明

子代理 · 技能 · 命令 · 工作流 · MCP 全景

本手册说明当前 ZCode 环境中自建的三类执行角色（子代理）、三个工作技能、六个斜杠命令、三个自动化工作流与三个 MCP 服务：它们各自解决什么问题、什么时候用、如何互相配合，以及模型成本如何分配、整套体系如何备份与迁移。

版本 1.0 · 2026 年 10 月

一、体系总览

这套体系解决的是同一个问题的不同环节：把“一句模糊的需求”变成“可验收的交付物”，并且在多任务、大项目场景下控制住模型成本。五个组成部分各司其职，串起来就是一条完整流水线。

3	3	6	3	3
子代理	技能	命令	工作流	MCP 服务

环节	组件	解决的问题
需求梳理	技能 brainstorming ＋ 命令 /brainstorm	需求模糊时，和用户一次一问地把目的、约束、成功标准问清楚
任务编排	技能 task-execute ＋ 命令 /task-execute	按任务形态自动路由：异构任务派子代理，同构批量走工作流
落地执行	子代理 coder（单个派遣）	把自包含的实现任务写成代码并自证验过
批量执行	工作流 swarm ＋ 命令 /swarm	一个模板扇出 N 个条目，并行子代理执行后统一汇总
执行验收	技能 task-review ＋ 命令 /task-review	把"声称做完"和"实际做没做"逐条对账
项目体检	工作流 project-review ＋ 命令 /project-review	对整个项目的架构、依赖、死代码、安全、文档做六维静态体检
质量门禁	工作流 quality-gate ＋ 命令 /quality-gate	按项目真实命令（typecheck/lint/test）跑门禁，退出码如实汇总
能力扩展	MCP：glm-vision / glm-websearch / glm-codegraph	给模型补上识图、联网搜索、代码知识图谱三类外部能力

三个执行角色（子代理）贯穿上述所有环节：

子代理	定位	典型触发	运行模型
coder	全栈实现工程师	“实现 / 修 bug / 补测试”类实施任务	deepseek-v4.1-flash
reviewer	专职评审员（只读）	改动复核、批量取证、体检维度	deepseek-v4.1-flash
researcher	综合调研员（零修改）	“查清来龙去脉 / 为什么 / 有哪些方案”	跟随子代理默认绑定

二、子代理：三类专业执行角色

子代理是"带着岗位纪律去干活"的独立执行体：每个子代理都从零上下文启动，因此派遣时任务说明必须自包含（输入、预期输出、完成判据）。

2.1 coder —— 全栈实现

把自包含的实现任务落成代码并验证。覆盖前端、后端、数据库、接口联调四个面：先读后改、最小 diff、改签名前全仓搜索调用方、验证命令必须从项目清单里确认存在、禁止擅自 git 提交与装依赖。

交付格式固定（做了什么 / 改动清单 file:line / 验证程度 / 未完成项），主代理据此验收。它是实施类任务的首选，只读调查不要派它。

2.2 reviewer —— 专职评审（只读）

只读取证，产出带证据的结构化评审报告。工具白名单只有 Read 与 Bash（只读命令），从结构上改不了文件。按六维度审查：要求达成的对账、正确性、漏改、副作用与回归、规范一致性、验证与回滚；每条结论必须带 file:line 或命令输出，证据分 A 已核实 / B 旁证 / C 无法核实三级。找不到问题也要列出查过哪里——用"没查"冒充"没问题"是禁止项。

2.3 researcher —— 综合调研（零修改）

三路取证：本地代码与文档检索、git 历史考古（log -S 追符号、blame 定位改动）、联网搜索（优先 glm-websearch）。产出带来源的调研报告，结论分"事实 / 推断 / 未查明"三档；未查明必须写明已尝试的路径与所缺条件，禁止拿"不清楚"当借口。对源项目零修改，临时整理只写 .research 目录。

2.4 与内置代理的分工

任务形态	派遣对象	原因
找文件 / 找代码位置 / 验证存在性	内置 Explore	快速定位，结构只读，成本最低
查清来龙去脉并要汇总报告	researcher	有 git 考古与网络三路取证、带诚实条款
实施实现	coder	带完整实现纪律与交付格式
改动复核 / 批量取证	reviewer	带证据分级与六维框架
不需要纪律约束的兜底任务	内置 general-purpose	全工具通用

三、技能：主代理的工作方法

技能不执行具体动作，而是给主代理注入一套工作方法；命中触发语时自动加载。

技能	解决的问题	何时触发
brainstorming	需求模糊时，一次一问澄清目的/约束/成功标准；遇到事实性未知派 researcher 查证，给出 2-3 个方案对比，分段确认设计后才动工	“我想加个 X” “帮我做个 Y” “一起梳理下需求”
task-execute	审计划 → 按形态路由 → 派遣 → 逐项验收 → 整体把关；异构任务派 coder/Explore，同构批量（≥3 项同模板独立任务）改走 swarm 工作流	“执行这个计划” “按方案落地”
task-review	把"我声称做了什么"与"代码实际是什么"逐条对账；校验用户原始要求覆盖、漏改、副作用、验证层级，发现属实测而非汇报	“复核一下刚才的改动” “真的做完了吗”

四、命令：六个斜杠入口

命令是薄入口：只负责把参数交给对应技能或工作流，本身不干活。

命令	参数	用途
/brainstorm	想法或需求描述	需求梳理对话，产出确认过的设计
/task-execute	任务或计划	编排执行：路由、派遣、验收、交付
/task-review	要复核的任务说明（可留空）	完成复核：声称 vs 事实对账
/project-review	体检范围路径（留空=整个工作区）	六维项目体检，落盘 REVIEW-日期.md
/quality-gate	留空自动探测；或指定命令	跑项目真实检查命令，退出码汇总
/swarm	任务描述	拆成独立子任务，批量并行执行后汇总

五、工作流：可复用的自动化执行体

工作流是保存在本机的脚本化执行体，参数化后可反复运行；每个工作流的子代理都在独立会话中执行，互不干扰，结束后统一汇总。

5.1 swarm —— 批量并行执行

一条含 {{item}} 占位符的模板 × 一个条目列表，每个条目一个子代理并行执行。参数：items（条目）、prompt_template（共享指令）、shared_context（公共背景）、persona（整批人设：coder / reviewer / researcher）、followups（对指定条目的同上下文追问）、output_mode（full/summary/status_only）、dry_run（只预览展开结果）。适合"给 N 个模块分别补测试""批量审查 N 个文件"这类同构任务；有依赖的子任务禁止放进同一批。

5.2 project-review —— 全项目体检

按维度并行只读取证，内置六维：架构与分层一致性、依赖健康、死代码与冗余、错误处理覆盖、安全基线、文档与注释过期（可自定义维度，最多 8 个）。每个维度一个审查员子代理，结果汇总并落盘 REVIEW-日期.md。参数：path（体检范围）、dimensions（维度列表）。它体检的是"项目当前状态"，复核"某次任务完成情况"请用 task-review——两者边界不要混。

5.3 quality-gate —— 质量门禁

逐条运行项目真实的检查命令（不传参数时自动探测 package.json 的 typecheck/lint/test 或 Makefile 目标，默认排除 build 类副作用命令），退出码按值汇总、不中断批次。参数：commands（[{tool, args, label}]，tool 限 npm/pnpm/yarn/dotnet/make/python/node/git）、timeout_ms。用于任务验收前的一键检查，或批量修复后的统一门禁。

六、MCP 服务：给模型补外部能力

三个 MCP 服务器都以标准 stdio 协议注册在本机，统一存放在 D:\Zcode\mcp 下，随 ZCode 启动自动连接。

服务	能力	工具	使用要点
glm-vision	识图：让不能看图的模型获得视觉理解	read_image（本地图片）、read_clipboard_image（剪贴板截图）	底层 MiniMax 视觉模型；思考型模型需给足输出上限；可分析报错截图、设计稿、图表
glm-websearch	联网搜索：中文与大陆网络环境友好	websearch（query / maxResults / lang）	四引擎自动降级（Bing → AnySearch → Exa → Parallel），失败时输出降级轨迹；自带引用来源；可选接入自建 SearXNG
glm-codegraph	代码知识图谱：符号级检索替代盲目 grep	find / impact / context / overview / cleanup	tree-sitter 解析（支持 C# 等 9 种语言），SQLite 每项目独立索引；impact 回答"改它会波及谁"；索引 7/30/60/90 天未用可清理

为什么值得配：识图让"没有视觉能力的模型"也能分析截图；搜索补齐实时信息；代码图谱把"改这个函数会影响谁"从全仓 grep 变成一次符号查询——三者都直接减少子代理的探索开销，从而控制 token 成本。

七、成本与额度控制

核心机制只有一条：子代理用哪个模型，以"子代理默认绑定"为唯一真相源，任何时候改绑定，整套体系自动跟随。

机制	行为	说明
单一真相源	agents-state.json 中的子代理默认绑定	在设置界面修改即可，所有 Agent 子代理派发时现读
工作流跟随	启动工作流前现读该绑定并作为 subagent_model 传入	保证工作流子代理与直接派遣的子代理用同一个模型；未配置绑定时才跟随会话模型
模型无关设计	工作流脚本不写死任何模型	换主代理、换子代理模型都不需要改任何脚本或配置
大任务开新会话	主代理上下文从零开始	长会话的上下文会随轮次放大成本，新任务新会话是最直接的节省手段
输出档位	swarm 默认 summary（截断 500 字）	大批量结果用 summary 或 status_only，完整证据看落盘文件

八、备份与迁移

全部自建资产集中在 D:\Zcode：mcp 目录是 MCP 服务器的生效位置，backup 目录是子代理、技能、命令、工作流与 MCP 快照的备份。迁移到新电脑时整目录拷贝，再按迁移指南铺设即可。

位置	内容	迁移方式
D:\Zcode\mcp\	三个 MCP 服务器实体（glm-vision / glm-websearch / glm-codegraph）	整目录拷贝；codegraph 需 npm install 一次
backup\agents\	coder / reviewer / researcher 定义	拷到新机 ~/.zcode/agents/
backup\skills\	brainstorming / task-execute / task-review	拷到新机 ~/.zcode/skills/
backup\commands\	六个斜杠命令	拷到新机 ~/.zcode/commands/

位置	内容	迁移方式
backup\workflows\	swarm / project-review / quality-gate	拷到新机 ~/.zcode/workflows/
backup\mcp\	MCP 快照（不含 node_modules 与索引缓存）	拷回 D:\Zcode\mcp 后 npm install
backup\AGENTS.md	全局工作约定（搜索优先级、批量路由、成本规则）	拷到新机 ~/.zcode/AGENTS.md

注意：供应商配置与 API Key 含在 ~/.zcode/cli/config.json 与 provider 配置中，刻意不放进备份包，迁移时需在新机重新配置（迁移指南有分步说明）。

九、使用场景速查

我想……	用这个
把一句模糊的想法变成明确设计	/brainstorm
按计划把任务做出来（自动决定派谁干）	/task-execute
检查刚才的改动到底做完没有	/task-review
给整个项目做一次静态体检	/project-review
跑项目的 typecheck / lint / test 看通不通	/quality-gate
把一批同构任务并行做完	/swarm
让模型看图 / 分析截图	glm-vision（read_image）
联网查资料（中文、带来源）	glm-websearch（websearch）
查符号在哪、改它会波及谁	glm-codegraph（find / impact）
把整套体系搬到新电脑	参照 backup/README.md 迁移指南